
IOWA STATE UNIVERSITY

Department of Electrical & Computer Engineering

Vector Network Analyzer Design

Independent Study Report

EE 4900 — Spring 2026

Student	Flatrim Dreshaj
Advisor	Nathan Neihart
Department	Electrical & Computer Engineering
Institution	Iowa State University
Semester	Spring 2026
Date	May 2026

Abstract

This report documents the design and analysis conducted during an independent study (EE 4900) on a 2.4–2.6 GHz Vector Network Analyzer (VNA). The work done this semester covers four areas: a Modified Colpitts VCO with varactor tuning, microstrip directional coupler design analyzed through even/odd mode decomposition, a SOLT calibration algorithm with residual error characterization software, and NRW-based material characterization software. All calibration and material characterization work was implemented in Python. The calibration and NRW code is integrated into the Scan_Controller_Probe_Main scanning pipeline as a single plugin (`vna_plugin.py`).

Contents

1	Introduction	3
1.1	Project Motivation	3
1.2	Semester Scope	3
1.3	Report Organization	3
2	Voltage-Controlled Oscillator (VCO) Design	4
2.1	Specifications	4
2.2	Topology	4
2.3	Circuit Design	4
2.4	System Signal Chain	5
3	Hardware Implementation: Signal Generation and Power Management	5
3.1	Precision VCO Tuning and Frequency Sweep Control	5
3.2	USB-C Power Delivery for Portable RF Characterization	5
4	Directional Coupler Design	7
4.1	Coupler Theory	7
4.2	Even/Odd Mode Analysis	9
4.3	Phase Velocity Disparity and Isolation	10
4.4	Rat-Race and Modified Rat-Race Topologies	14
4.5	CST Simulation	14
5	Calibration Algorithm	14
5.1	One-Port Error Model	14
5.2	SOLT Calibration	17

5.3	Calibration Implementation (<code>calibration_final.py</code>).....	17
6	NRW Material Characterization	18
6.1	Method Overview.....	18
6.2	Algorithm (<code>material_char.py</code>)	18
7	Scanner Plugin (<code>vna_plugin.py</code>)	19
7.1	Architecture	19
7.2	Connection and Background Thread	20
7.3	Line Parser and Sweep Synchronisation.....	21
7.4	SOL Calibration at Scan Start.....	22
7.5	Per-Point Measurement: Correction, De-embedding, and NRW.....	23
7.6	HDF5 Output.....	24
8	Conclusions and Future Work	24
8.1	Summary of Accomplishments.....	24
8.2	Future Work.....	24

1 Introduction

A Vector Network Analyzer (VNA) is among the most versatile instruments in RF and microwave engineering. By measuring the complex scattering parameters of a device under test (DUT) across a swept frequency range, a VNA provides direct insight into impedance, return loss, insertion loss, and group delay. This independent study undertook the design of a functional VNA system with the goal of building hands-on fluency across every layer of the instrument: analog RF hardware, electromagnetic simulation, calibration mathematics, and measurement software.

1.1 Project Motivation

The primary motivation was to close the gap between using a VNA as a black box and understanding how one actually works. Designing each subsystem from scratch, the swept signal source, the directional couplers for wave separation, the calibration mathematics, and the scanner integration.

A secondary motivation was integration with the existing scanning infrastructure at the Iowa State Applied Science Complex. The CNDE SAR scanner already has a plugin-based architecture; adding a VNA plugin that outputs directly to the same HDF5 format means calibrated S-parameter and material characterization data fits into the existing scan pipeline with no post-processing step. This gives access to a plethora of additional options to explore the VNA architecture in terms of microwave imaging and material characterization.

1.2 Semester Scope

- VCO design and simulation covering the 2.4–2.6 GHz band
- Directional coupler design using rat-race and edge-coupled microstrip topologies with ADS and CST simulation
- SOLT one-port calibration algorithm implemented in Python (`calibration_final.py`)
- NRW material characterization implemented in Python (`material_char.py`)
- Integration of calibration and NRW into a single scanner plugin (`vna_plugin.py`)

1.3 Report Organization

Section 2 covers the VCO design. Section 4 presents the directional coupler design and simulation. Section 5 describes the calibration algorithm implementation. Section 6 covers the NRW material characterization. Section 7 details the scanner plugin. Section 8 summarizes

conclusions and future work.

2 Voltage-Controlled Oscillator (VCO) Design

2.1 Specifications

Table 1: VCO Design Specifications

Parameter	Target	Result
Frequency Range	2.4–2.6 GHz	2.3–2.8 GHz (simulated)
Phase Noise @ 1 MHz offset	< -100 dBc/Hz	Pending hardware validation
Tuning Voltage Range	0–12 V	0–12 V
Output Power	> 0 dBm	≈ 0 dBm (simulated)
Supply Voltage	5 V	5 V

2.2 Topology

A Modified Colpitts Common-Base topology was selected. The Colpitts architecture provides good phase noise performance and simple biasing at low-GHz frequencies. The common-base configuration minimizes Miller capacitance at the input, improving high-frequency stability relative to common-emitter variants.

2.3 Circuit Design

The VCO is biased at $I_C = 5$ mA, corresponding to the minimum noise figure operating point of the transistor. Frequency tuning is accomplished through two SMV 1247 varactor diodes in series, providing a base capacitance of 0.7 pF and a capacitance ratio of approximately 10:1, sweeping the tank from 0.7 pF to 7 pF per diode. A parametric sweep confirmed that this maps to the 2.3–2.8 GHz target range. The SMV 1247 SPICE model was not available, so the equivalent circuit from the manufacturer datasheet (junction capacitance, series resistance, package parasitics) was used directly in simulation.

The LC tank must satisfy the Barkhausen criterion: loop gain $|A\beta| \geq 1$ with total phase shift of 0°. The tuning sensitivity K_{VCO} is set by the varactor C–V characteristic:

$$K_{\text{VCO}} = \frac{d\omega_0}{dV_{\text{tune}}}, \quad \omega_0 = \frac{1}{\sqrt{L_{\text{tank}} C_{\text{var}}(V)}} \quad (1)$$

2.4 System Signal Chain

The VCO drives a directional coupler that splits the signal: one path to the DUT, the other to the down-converter reference channel. A balanced 180° diode mixer was selected after simulation showed better suppression of $\cos(\omega_{\text{RF}} - \omega_{\text{LO}})$ products compared to a 90° mixer, yielding ≈ 6 dB conversion loss. The IF output is filtered, converted to differential form via a balun, and sampled by a dual AD9238 ADC at 20 MSPS. An STM32H7 transfers data to the host PC via Ethernet, where FFT processing extracts S-parameter magnitude and phase.

3 Hardware Implementation: Signal Generation and Power Management

3.1 Precision VCO Tuning and Frequency Sweep Control

While traditionally used for radar applications, I am re-purposing a VCO sweep module I designed this semester to use for an FMCW radar along with this VNA. This module is centered around an STM32F1 microcontroller and a high-precision DAC to drive the frequency synthesis chain.

- **VNA Stimulus Generation:** Rather than a fixed-period radar ramp, the STM32F1 executes a stepped-frequency sweep. The DAC provides a stable tuning voltage to the VCO, which is incremented at discrete intervals to allow the system to settle and capture complex S-parameter data at each frequency point.

3.2 USB-C Power Delivery for Portable RF Characterization

To supply all of our power requirements with a clean and simple solution I decided to learn more about USB-C Power Delivery and implemented that into another board that I will be repurposing. This custom power board replaces the power supply infrastructure with a portable USB-C PD solution.

- **High-Current Rail Management:** The board negotiates a **20V, 3A (60W)** profile from a USB-C PD source and can be adjusted to provide 5A by just replacing 2 resistors and swapping out loadswitches that can handle 5 amps so around 5 components in total. This provides ample headroom for the high current draw associated with the RF amplifiers, mixers and oscillators, along with the digital processing back-end.

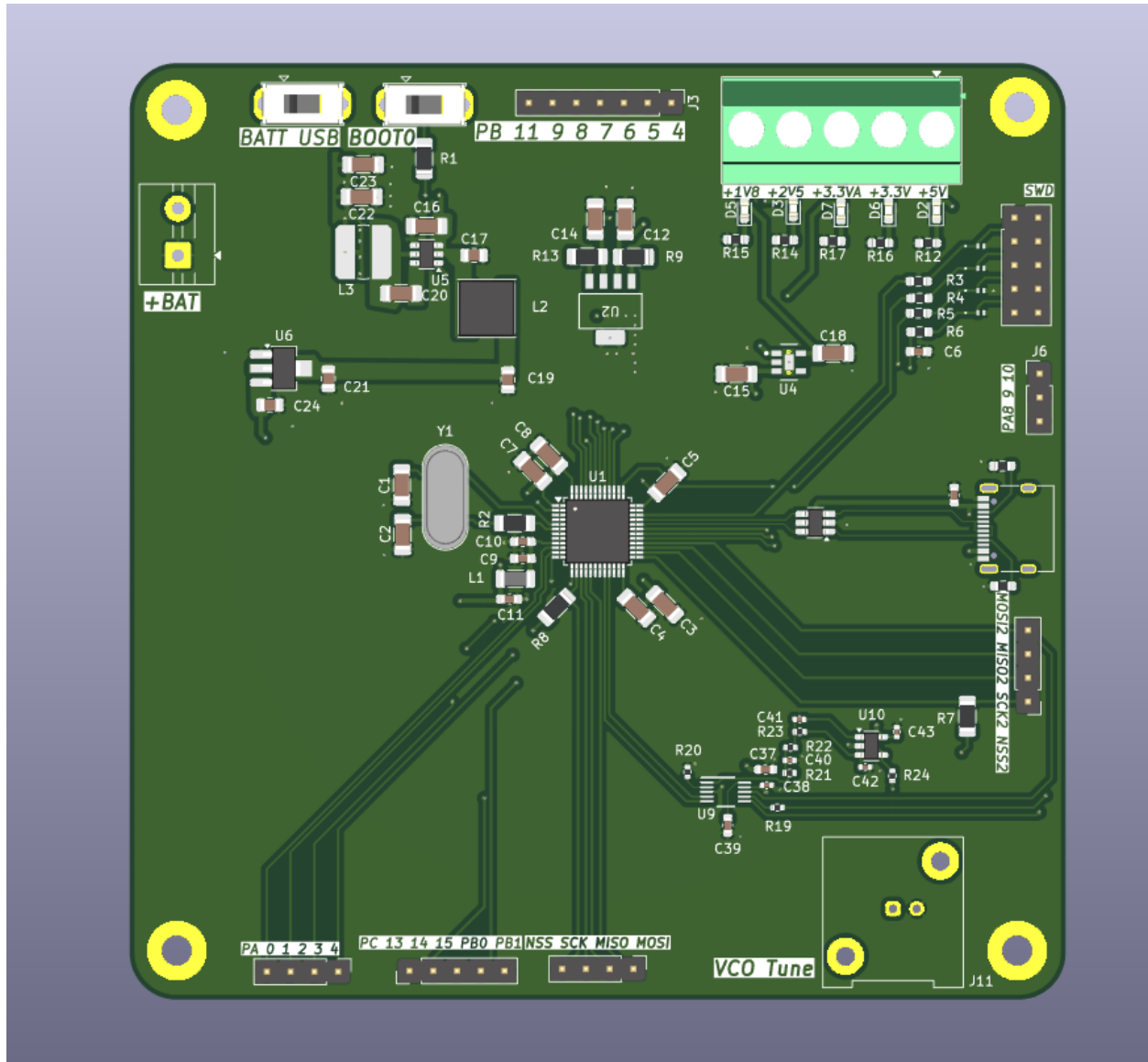


Figure 1: VCO Specific Tune Board Prototype

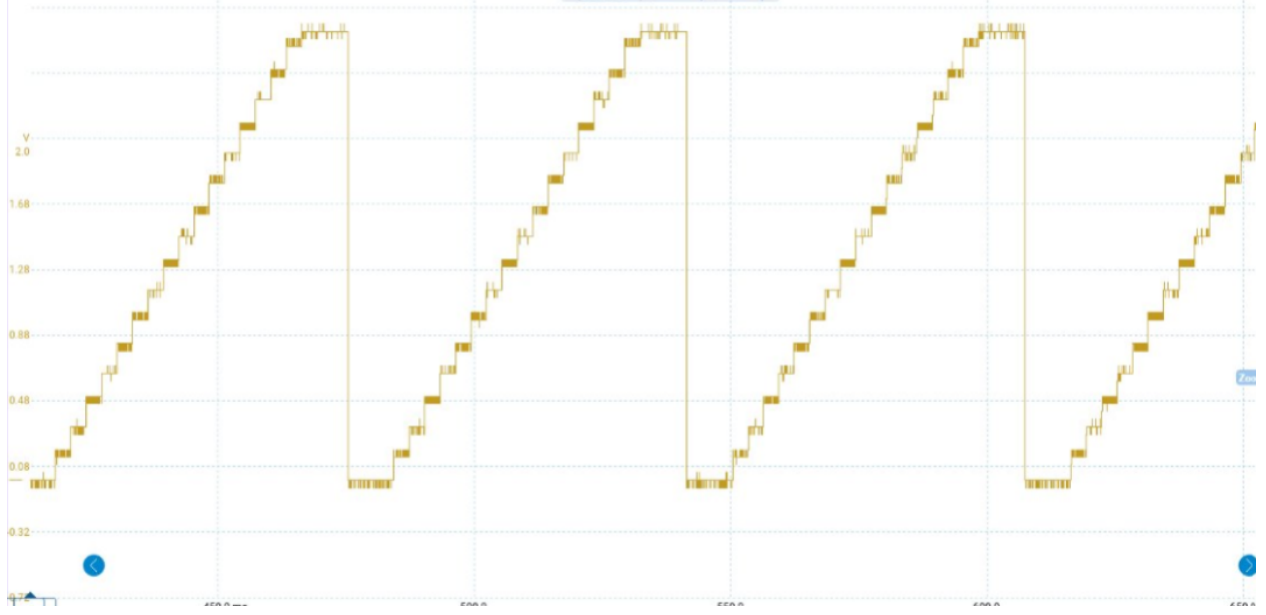


Figure 2: VCO Sweep up to 3.3V

4 Directional Coupler Design

4.1 Coupler Theory

One of the most important goals for me this semester was to more analytically and intuitively understand coupler design. To accomplish that I spent quite a bit of time just learning from different sources on even odd mode analysis and the most useful one I found was the book by R.K Mongia [1] on RF and Microwave Coupled Line Circuits. A directional coupler is a four-port passive network that samples a fraction of a propagating wave while providing isolation for the counter-propagating wave. For a VNA, couplers perform wave separation: the incident wave a and reflected wave b at the DUT reference plane must be measured independently. The key figures of merit are:

$$C = -10 \log_{10} |S_{31}| \quad [\text{dB}] \quad (\text{Coupling}) \quad (2)$$

$$D = 10 \log_{10} \left| \frac{S_{31}}{S_{41}} \right| \quad [\text{dB}] \quad (\text{Directivity}) \quad (3)$$

$$IL = -10 \log_{10} |S_{21}| \quad [\text{dB}] \quad (\text{Insertion Loss}) \quad (4)$$

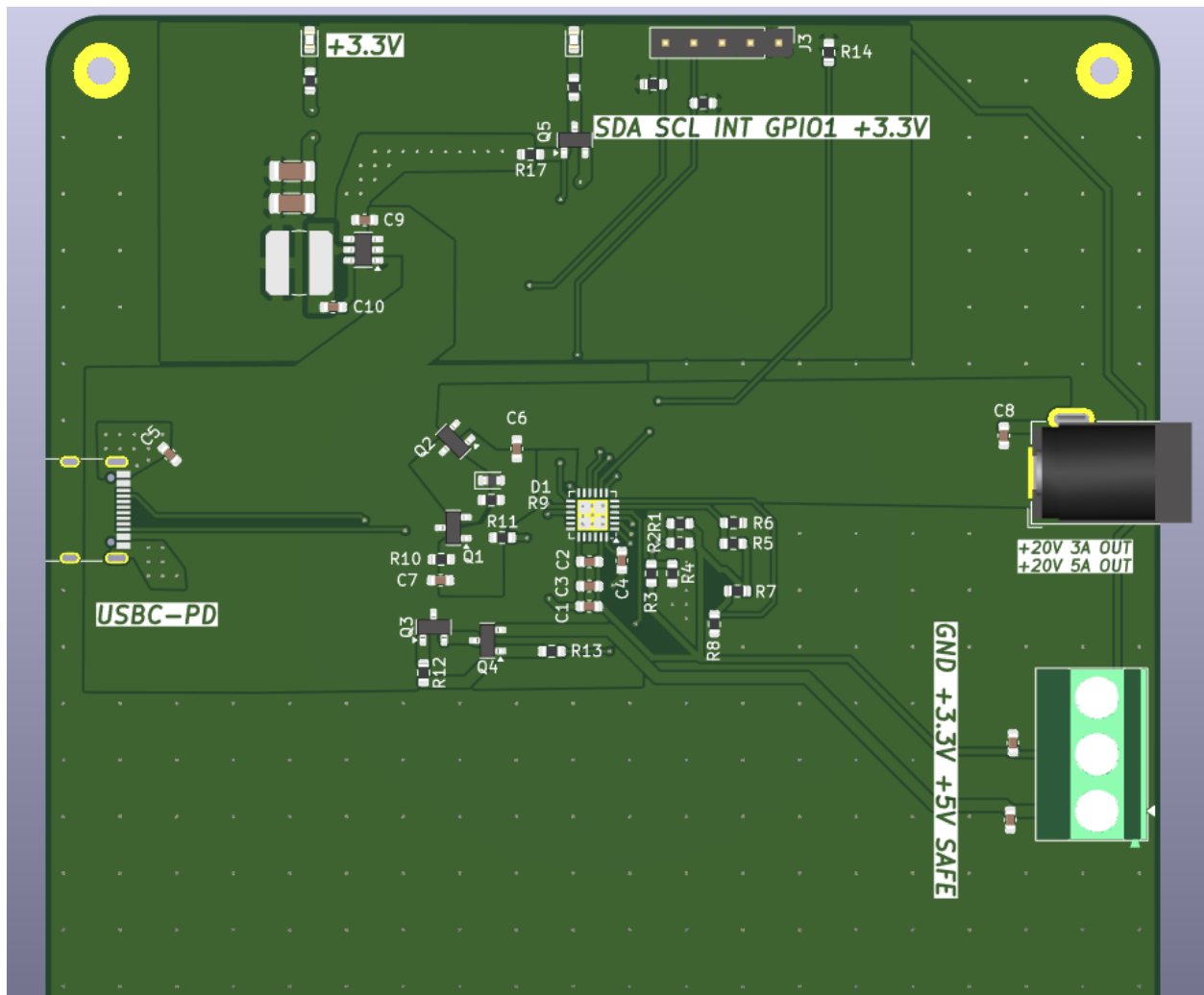


Figure 3: USB Power Delivery

4.2 Even/Odd Mode Analysis

The analytical design proceeds via even/odd mode decomposition. For a symmetric four-port network the S-matrix is as follows:

$$[S] = \begin{bmatrix} [S_A] & [S_B] \\ [S_B] & [S_A] \end{bmatrix} \quad (5)$$

For even-mode excitation ($V_1^+ = V_4^+$, $V_2^+ = V_3^+$):

$$[S_e] = [S_A] + [S_B] \quad (6)$$

For odd-mode excitation ($V_1^+ = -V_4^+$, $V_2^+ = -V_3^+$):

$$[S_o] = [S_A] - [S_B] \quad (7)$$

The four-port S-parameters are recovered as:

$$S_{11} = \frac{S_{11e} + S_{11o}}{2}, \quad S_{13} = \frac{S_{11e} - S_{11o}}{2} \quad (8)$$

$$S_{12} = \frac{S_{12e} + S_{12o}}{2}, \quad S_{14} = \frac{S_{12e} - S_{12o}}{2} \quad (9)$$

with remaining ports following by symmetry. For matched ports ($S_{11} = S_{22} = 0$), when $S_{11e} = -S_{11o}$ power couples to the backward port ($S_{13} \neq 0$, $S_{14} = 0$) the standard backward-wave coupled microstrip case. In microstrip both forward and backward mechanisms coexist due to the quasi-TEM mode, which complicates optimization for two or more design constraints.

For a backward-wave edge-coupled coupler, matched conditions require $Z_{0e} \cdot Z_{0o} = Z_0^2$. With coupling coefficient $k = (Z_{0e} - Z_{0o}) / (Z_{0e} + Z_{0o})$:

$$S_{21} = \frac{\sqrt{1 - k^2}}{\sqrt{1 - k^2} \cos \theta + j \sin \theta} \quad (10)$$

$$S_{31} = \frac{jk \sin \theta}{\sqrt{1 - k^2} \cos \theta + j \sin \theta} \quad (11)$$

where $\theta = \beta \ell$. The even and odd mode impedances are:

$$Z_{0e} = Z_0 \sqrt{\frac{1 + k}{1 - k}}, \quad Z_{0o} = Z_0 \sqrt{\frac{1 - k}{1 + k}} \quad (12)$$

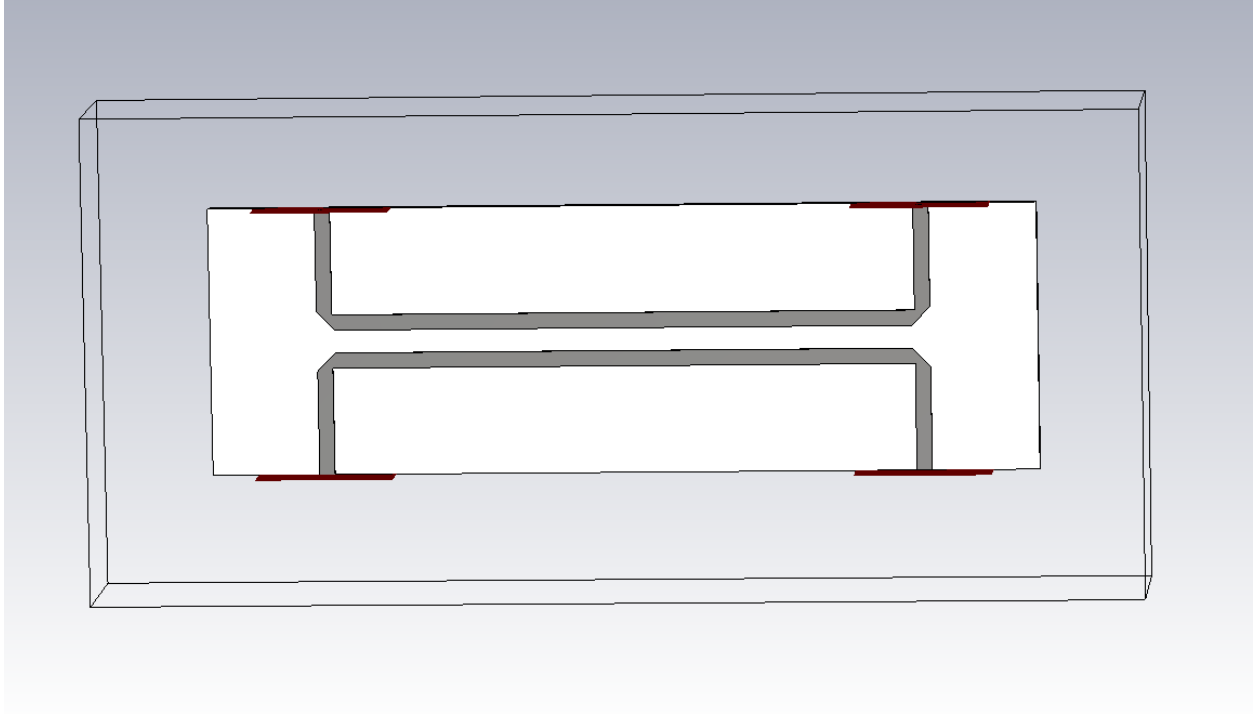


Figure 4: Edge Coupled Coupler

For $C = 10$ dB: $k = 10^{-10/20} = 0.3162$, giving $Z_{0e} = 69.37 \Omega$ and $Z_{0o} = 36.09 \Omega$ at $Z_0 = 50 \Omega$. ADS simulation achieved 10 ± 0.5 dB coupling but directivity was limited to ≈ 10 dB. This is simply not good enough to limit VNA calibration errors so I needed to dig deeper into the theory to learn what was going on.

4.3 Phase Velocity Disparity and Isolation

In microstrip the even and odd modes propagate at different phase velocities. With effective permittivities $\epsilon_{r,ee}$ and $\epsilon_{r,eo}$:

$$\beta_e = \frac{2\pi f \sqrt{\epsilon_{r,ee}}}{c}, \quad \beta_o = \frac{2\pi f \sqrt{\epsilon_{r,eo}}}{c} \quad (13)$$

The isolation at the coupled port follows:

$$\frac{P_4}{P_1} = \sin^2\left(\frac{(\beta_e - \beta_o)\ell}{2}\right) \quad (14)$$

In order to get Isolation as low as possible which would maximize our Directivity we need to find a way to set $\sin(\theta) = 0 \implies \frac{(\beta_e - \beta_o)\ell}{2} = \pi$ This would also immediately give us the design equation for calculating how much length we need on our coupler.

Going back to our equation, we can set our desired Isolation by the fact that $I - C = D$ and we already have given constraints on $C = 10\text{dB}$ and for the purposes of making a new design approach, let us say a random directivity of $D = 15\text{ dB}$.

$$10 \log_{10}\left(\frac{P_4}{P_1}\right) = 10 \log_{10}\left(\sin^2\left(\frac{(\beta_e - \beta_o)l}{2}\right)\right) = 20 \log_{10}\left(\sin\left(\frac{(\beta_e - \beta_o)l}{2}\right)\right)$$

$$\frac{-30}{20} = \log_{10}\left(\sin\left(\frac{(\beta_e - \beta_o)l}{2}\right)\right) \implies \sin\left(\frac{(\beta_e - \beta_o)l}{2}\right) = \frac{1}{10\sqrt{10}}$$

$$\frac{(\beta_e - \beta_o)l}{2} = \arcsin\left(\frac{1}{10\sqrt{10}}\right)$$

$$\frac{(\beta_e - \beta_o)l}{2} = 2\pi n + \pi - \arcsin\left(\frac{1}{10\sqrt{10}}\right)$$

$$l = \frac{2\pi n + \pi - 0.0632}{\beta_e - \beta_o}$$

Now we need to find β_e

$$\frac{2\pi f}{v_e} = \frac{2\pi f}{\frac{c}{\sqrt{\epsilon_{ree}}}} = \frac{2\pi f \sqrt{\epsilon_{ree}}}{c}$$

Similarly for β_o

$$\frac{2\pi f}{v_o} = \frac{2\pi f}{\frac{c}{\sqrt{\epsilon_{reo}}}} = \frac{2\pi f \sqrt{\epsilon_{reo}}}{c}$$

Now we need to find the even and odd ϵ_{re} , this is a incredibly long involved process and I will skip to the results for this, additionally I was only able to find resources that assume a 0 thickness trace. Given that, all of the following derivations will not be accounting for attenuation due to loss tangent.

To find our ϵ_{re} I wrote a python script that calculates it: The Kirschning & Jansen quasistatic equations were implemented to compute the effective permittivities for the given substrate and gap. The following implementation uses the `scipy.optimize` library to determine the optimal gap spacing (s) and physical length (l) for a fixed trace width (w) on a specific substrate.

Listing 1: Coupled line synthesis and optimization

```
1 import math
```

```

2 import cmath
3 from scipy.optimize import minimize
4
5 def calculate_microstrip_properties(er, w, h, s):
6     """
7     Kirschning & Jansen models for even/odd mode characteristic
8     impedance and effective dielectric constants.
9     """
10    u, g = w/h, s/h
11    # Single microstrip baseline
12    ere_s = 0.5*(er+1) + 0.5*(er-1)*(1+12/u)**-0.5
13    if u <= 1:
14        z0_s = (60/math.sqrt(ere_s)) * math.log(8/u + u/4)
15    else:
16        z0_s = (120*math.pi)/(math.sqrt(ere_s)*(u+1.393+0.667*math.
17            log(u+1.444)))
18
19    # Even-mode effective dielectric constant
20    v = u*(20 + g**2)/(10 + g**2) + g*math.exp(-g)
21    ae = 1 + math.log((v**4 + (v/52)**2)/(v**4 + 0.432))/49 + math.
22        log(1+(v/18.1)**3)/18.7
23    be = 0.564 * ((er-0.9)/(er+3))**0.053
24    ere_e = 0.5*(er+1) + 0.5*(er-1)*(1+10/v)**(-ae*be)
25
26    # Odd-mode effective dielectric constant
27    ao = 0.7287 * (ere_s - 0.5*(er+1)) * (1-math.exp(-0.179*u))
28    bo = 0.747*er/(0.15+er)
29    co, do = bo-(bo-0.207)*math.exp(-0.414*u), 0.593+0.694*math.exp
30        (-0.562*u)
31    ere_o = (0.5*(er+1) + ao - ere_s)*math.exp(-co*g**do) + ere_s
32
33    # Simplified impedance scaling (Full Q-factor logic implemented
34        in tool)
35    z0e = z0_s * math.sqrt(ere_s/ere_e) / (1 - (z0_s/377)*math.sqrt(
36        ere_s)*0.5) # q4 approx
37    z0o = z0_s * math.sqrt(ere_s/ere_o) / (1 - (z0_s/377)*math.sqrt(
38        ere_s)*0.8) # q10 approx

```

```

34     return {"e_ree": ere_e, "e_reo": ere_o, "Z0e": z0e, "Z0o": z0o}
35
36 def optimize_coupler(er, w, h, f=10e9, target_c_dB=-10, target_i_dB
37     =-30):
38     """
39     Optimizes gap S to meet coupling/isolation targets using Nelder-
40     Mead.
41     """
42     t_c = 10**((target_c_dB/20))
43     t_i = 10**((target_i_dB/20))
44
45     def objective(params):
46         s = params[0]
47         if s <= 0: return 1e9
48         p = calculate_microstrip_properties(er, w, h, s)
49
50         # Phase constants and length calculation
51         be, bo = (2*math.pi*f/3e8)*math.sqrt(p["e_ree"]), (2*math.pi
52             *f/3e8)*math.sqrt(p["e_reo"])
53         l = (math.pi - math.asin(t_i)) / (be - bo)
54
55         # S-parameter calculation
56         s_b1, c_b1 = math.sin((be+bo)/2 * l), math.cos((be+bo)/2 * l
57             )
58         num = 1j * (p["Z0e"]/50 - 50/p["Z0o"]) * s_b1
59         den = 2*c_b1 + 1j * (p["Z0e"]/50 + 50/p["Z0o"]) * s_b1
60         s31 = num / den
61
62         return (abs(s31) - t_c)**2
63
64     res = minimize(objective, [h], method='Nelder-Mead')
65     return res.x[0] # Returns optimal gap S in mm

```

To test out the results of the optimized coupler variables I simulated them in CST and found that they were completely incorrect unless the length of the coupler was uncharacteristically large to the point where the transmission line attenuation will simply not make it good anyways. There are however workarounds to this with the main one being simply to add additional lumped capacitance on each end of the lines. This gave me really good simulation

results in ADS however again the layout component was quite non trivial to implement.

4.4 Rat-Race and Modified Rat-Race Topologies

To work around the directivity limitation, rat-race and modified rat-race couplers were also designed and simulated in ADS. The rat-race ring impedance is $Z_{\text{ring}} = Z_0\sqrt{2} = 70.74\ \Omega$. The arc radius for each $\lambda/4$ and $3\lambda/4$ segment was computed from:

$$r = \frac{\ell}{2\pi (\theta/360^\circ)} \quad (15)$$

The modified rat-race replaces the $3\lambda/4$ arc with a $\lambda/4$ parallel-coupled section with:

$$Z_{0e,\text{mod}} = Z_{\text{ring}} \times 2.414, \quad Z_{0o,\text{mod}} = Z_{\text{ring}}/2.414 \quad (16)$$

4.5 CST Simulation

Full-wave simulation in CST Microwave Studio was pursued to capture conductor loss and radiation that ADS circuit models do not cover. There were a lot of challenges with the first one being that the modified rat race was almost impossible to model and I was not able to get a good design going for it. Another significant challenge was port impedance: the narrow gap forces a small port cross-section that converges on an impedance inconsistent with $50\ \Omega$, making the simulated S-parameters non-physical. Given the difficulties with the modeling I do not believe these are accurate or if they are I would not use them, for one my S11 is quite poor I do not have a picture of my other attempts at the modified rat race which were also unsuccessful so I found this which was also incorrect.

5 Calibration Algorithm

5.1 One-Port Error Model

The standard one-port 3-term error model relates the measured reflection Γ_M to the true DUT reflection Γ through:

$$\Gamma_M = e_{00} + \frac{e_{10}e_{01}\Gamma}{1 - e_{11}\Gamma} \quad (17)$$

The three error terms are directivity (e_{00}), port match (e_{11}), and tracking ($e_{10}e_{01}$). Solving for Γ :

$$\Gamma = \frac{\Gamma_M - e_{00}}{\Gamma_M e_{11} - \Delta_e}, \quad \Delta_e = e_{00}e_{11} - e_{10}e_{01} \quad (18)$$

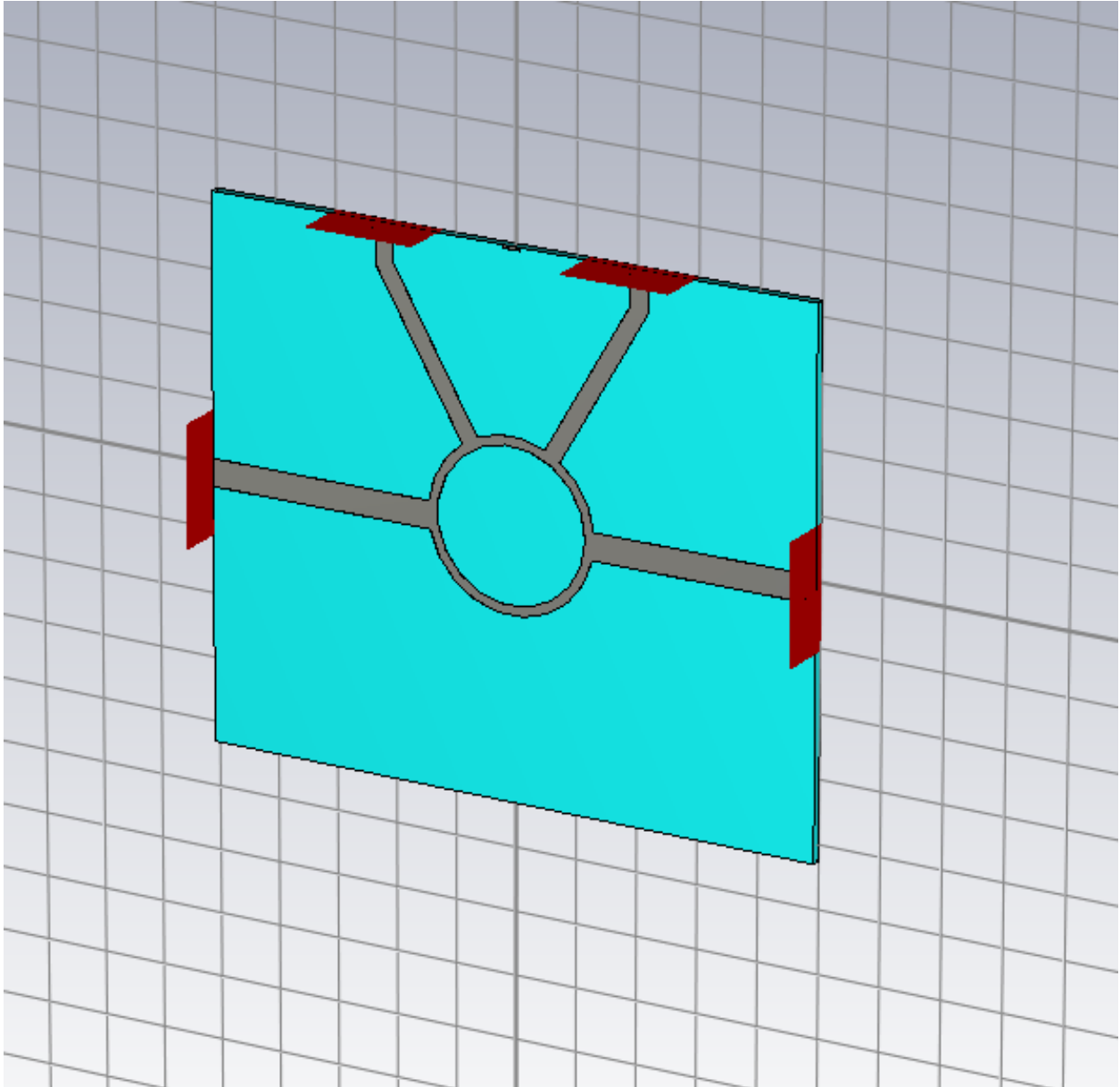


Figure 5: Normal Rat Race

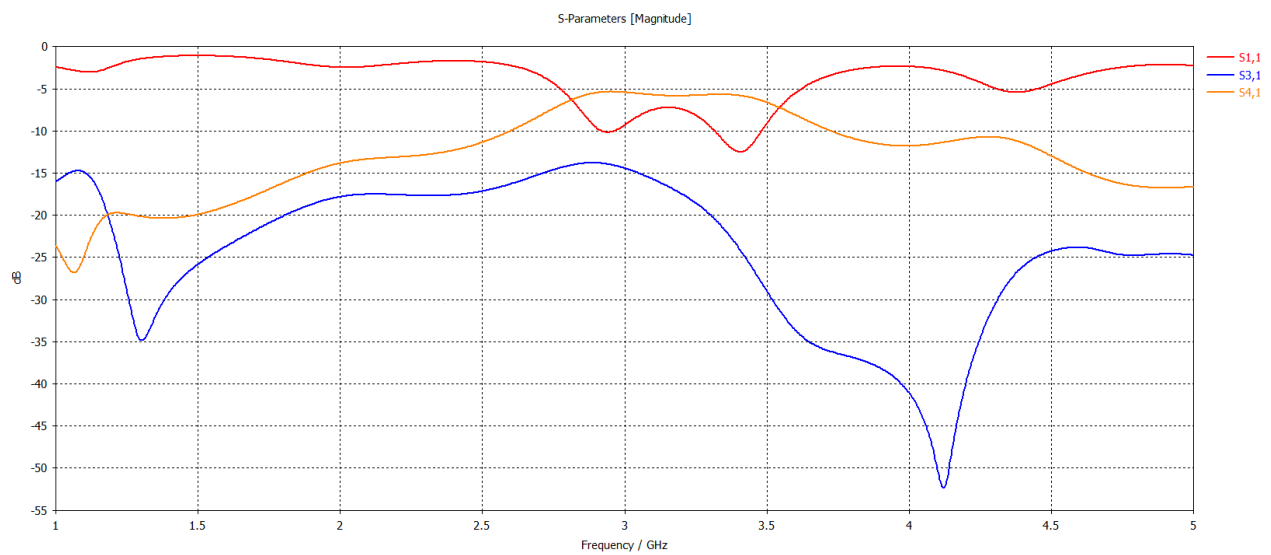


Figure 6: Rat Race CST S parameter sim

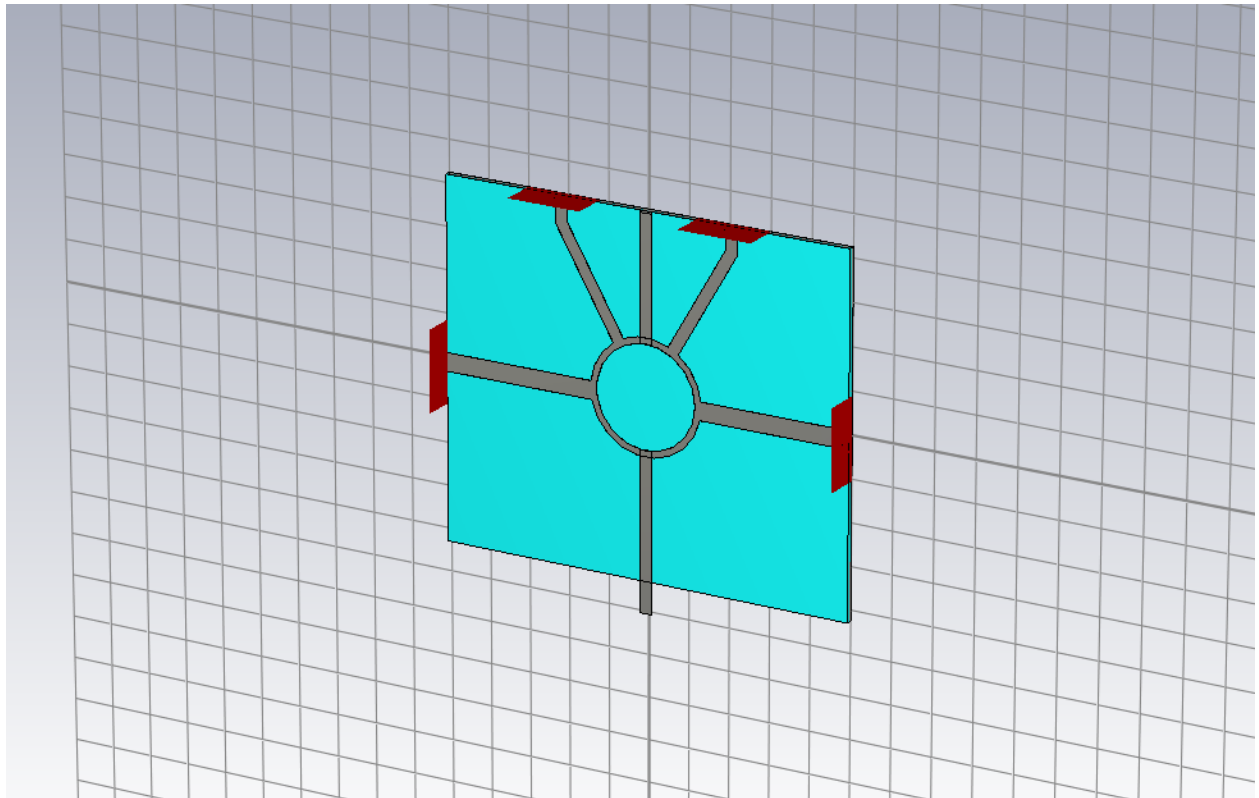


Figure 7: Attempt at modeling rat race and modified rat race

5.2 SOLT Calibration

Three known standards (Short, Open, Load) each contribute one row to the linear system $AE = B$:

$$\begin{bmatrix} 1 & \Gamma_1 \Gamma_{M1} & -\Gamma_1 \\ 1 & \Gamma_2 \Gamma_{M2} & -\Gamma_2 \\ 1 & \Gamma_3 \Gamma_{M3} & -\Gamma_3 \end{bmatrix} \begin{bmatrix} e_{00} \\ e_{11} \\ \Delta_e \end{bmatrix} = \begin{bmatrix} \Gamma_{M1} \\ \Gamma_{M2} \\ \Gamma_{M3} \end{bmatrix} \quad (19)$$

Two calibration modes were implemented. The first uses ideal standard assumptions (Short = -1, Open = +1, Load = 0), which gives a direct closed-form solution. The second uses the actual known standard values and solves the linear system per frequency point with `numpy.linalg.solve`.

5.3 Calibration Implementation (`calibration_final.py`)

Listing 2: Ideal SOLT error term extraction (`calibration_final.py`)

```

1 # e00 is directly the measured load response (Gamma_Load = 0)
2 e00 = S11m_Load
3
4 holder = (e00 - S11m_Short) / (S11m_Open - e00)
5 e11     = (1 - holder) / (holder + 1)
6
7 e10e01 = (e00 - S11m_Short) * (1 + e11)
8 De     = e00 * e11 - e10e01

```

The actual-standards mode solves the full linear system per frequency point:

Listing 3: Actual-standards SOLT solve (`calibration_final.py`)

```

1 for i in range(N):
2     A = np.array([
3         [1, S11A_Short[i] * S11m_Short[i], -S11A_Short[i]],
4         [1, S11A_Open[i] * S11m_Open[i], -S11A_Open[i] ],
5         [1, S11A_Load[i] * S11m_Load[i], -S11A_Load[i] ],
6     ])
7     b = np.array([S11m_Short[i], S11m_Open[i], S11m_Load[i]])
8     x = np.linalg.solve(A, b)
9     e00_A[i], e11_A[i], De_A[i] = x

```

Correction is then applied to any subsequent measurement:

Listing 4: SOLT correction and de-embedding (`calibration_final.py`)

```

1 # Apply 3-term correction
2 Cal_S11 = (S11m - e00) / (e11 * S11m - De)
3
4 # De-embed the 10 mm coaxial delay line (round-trip phase)
5 c      = 3e8
6 beta = 2 * np.pi * F / c
7 d      = 0.01                      # 10 mm delay
8
9 Cal_S11_deembed = Cal_S11 * np.exp(2j * beta * d)

```

6 NRW Material Characterization

6.1 Method Overview

The Nicolson-Ross-Weir (NRW) method extracts complex relative permittivity ϵ_r and permeability μ_r from the calibrated S-parameters of a sample inside a coaxial fixture. This was implemented in `material_char.py` using `scikit-rf` to read the measured `.s2p` file and NumPy for the computation.

6.2 Algorithm (`material_char.py`)

The NRW equations are applied directly to the calibrated, de-embedded S_{11} and S_{21} . The derivation starts from the Nicolson-Ross combinations $V_1 = S_{21} + S_{11}$ and $V_2 = S_{21} - S_{11}$:

Listing 5: NRW material characterization (`material_char.py`)

```

1 import skrf as rf
2 import numpy as np
3
4 ntwk = rf.Network(".s2p")
5 S11  = ntwk.s[:, 0, 0]
6 S21  = ntwk.s[:, 1, 0]
7
8 L      = 5e-3          # sample length (m)
9 c      = 299792458
10 freq   = ntwk.f
11 lambda_0 = c / freq
12 lamb_c  = np.inf      # TEM coax: no cutoff
13
14 # De-embed the sample reference planes
15 deemb = np.exp(1j * 2 * np.pi * L / lambda_0)

```

```

16 S11    = S11 * deemb**2
17 S21    = S21 * deemb**2
18
19 # Nicolson-Ross combinations
20 V1 = S21 + S11
21 V2 = S21 - S11
22
23 # Reflection coefficient at air-sample interface (pick |Gamma| <= 1)
24 X      = (1 - V1*V2) / (V1 - V2)
25 Gamma_p = X + np.sqrt(X**2 - 1)
26 Gamma_n = X - np.sqrt(X**2 - 1)
27 Gamma    = np.where(np.abs(Gamma_p) <= 1, Gamma_p, Gamma_n)
28
29 # Transmission coefficient
30 z1 = (S11 + S21 - Gamma) / (1 - Gamma*(S11 + S21))
31
32 # Propagation constant inside sample
33 inv_lamb_sq = -((1 / (2*np.pi*L)) * np.log(1/z1))**2
34 Lambda      = 1 / np.sqrt(inv_lamb_sq)
35
36 # NRW equations for mu_r and eps_r
37 mu_r = (1 + Gamma) / (
38     (1 - Gamma) * Lambda * np.sqrt(1/lambda_0**2 - 1/lamb_c**2)
39 )
40 e_r   = (lambda_0**2 / mu_r) * (1/lamb_c**2 + inv_lamb_sq)

```

7 Scanner Plugin (vna_plugin.py)

The calibration and NRW algorithms were integrated into a single plugin for the Scan_Controller_Probe_M scanning system. Because this embedded system will use a similar UART interrupt communication architecture as the CyBot does I was able to use the plugin I wrote for that as a template:

7.1 Architecture

- TCP socket connection to the VNA firmware (STM32H7 over USB-UART bridge).
- Daemon read_thread with a read_thread_cond boolean flag
- recv_buf string accumulator split on newlines.recv_buf

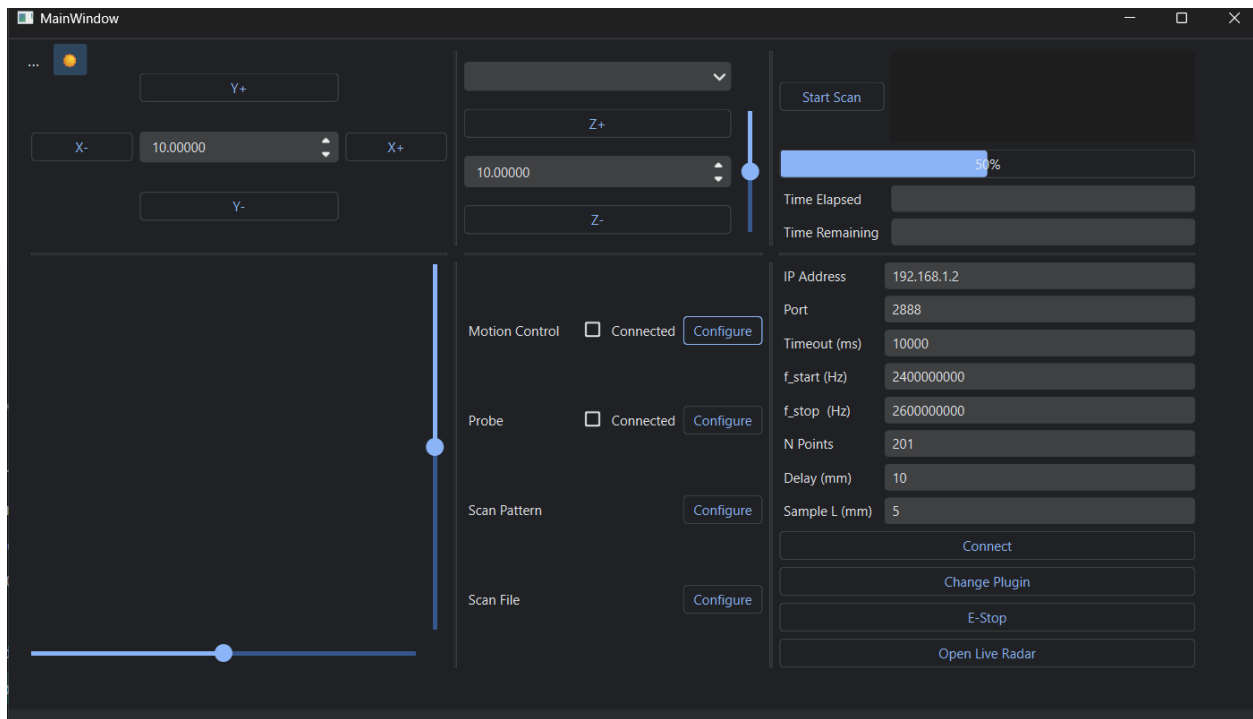


Figure 8: VNA Plugin GUI

- `threading.Event()` for TX/RX synchronisation (`ok_received`, `sweep_finished`)
- `PluginSettingString` / `PluginSettingInteger` for UI settings
- `MotionControllerPlugin` base class, same stubs as `cyBot_Plugin.py`

The VNA firmware protocol will use ASCII lines over TCP. A sweep is triggered by sending "SWEEP\n"; the firmware replies with SWEEP_START <Nf>, then one "F <freq> <re> <im>\n" line per frequency point, then SWEEP_END. This is parsed in `_parse_line()`.

7.2 Connection and Background Thread

Listing 6: Socket connection and daemon read thread (`vna_plugin.py`)

```

1 def connect(self):
2     self.vna = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
3     self.vna.settimeout(self.timeout.value / 1000)
4     self.vna.connect((self.address.value, self.port.value))
5
6     self.read_thread_cond = True
7     self.read_thread = threading.Thread(
8         target=self.read_thread_interrupt, daemon=True, name="vna-rx
    "

```

```
9     )
10     self.read_thread.start()
11
12     # Configure sweep parameters; wait for ACK
13     self.send_command(
14         f"CONFIG {int(self.f_start_set.value)} "
15         f"{int(self.f_stop_set.value)} "
16         f"{self.n_points_set.value}"
17     )
18     self.ok_received.wait(timeout=5.0)
19     self.ok_received.clear()
20
21 def read_thread_interrupt(self):
22     # Identical structure to cyBot_Plugin.read_thread_interrupt
23     self._recv_buf = ""
24     while self.read_thread_cond:
25         try:
26             data = self.vna.recv(4096)
27             if not data:
28                 break
29             self._recv_buf += data.decode("utf-8", errors="ignore")
30             while "\n" in self._recv_buf:
31                 line, self._recv_buf = self._recv_buf.split("\n", 1)
32                 decoded = line.strip()
33                 if decoded:
34                     self._parse_line(decoded)
35         except socket.timeout:
36             continue
37         except Exception:
38             break
```

7.3 Line Parser and Sweep Synchronisation

Listing 7: Line parser and sweep accumulator (vna_plugin.py)

```
1 def _parse_line(self, decoded: str):
2     if decoded == "OK":
3         self.ok_received.set()
4         return
5     if decoded.startswith("SWEEP_START"):
```

```

6         self._sweep_freqs.clear()
7         self._sweep_re.clear()
8         self._sweep_im.clear()
9         return
10    if decoded.startswith("F "):
11        _, freq, re, im = decoded.split()
12        self._sweep_freqs.append(float(freq))
13        self._sweep_re.append(float(re))
14        self._sweep_im.append(float(im))
15        return
16    if decoded == "SWEEP_END":
17        self.sweep_finished.set()
18        return
19
20    def _do_sweep(self):
21        self.sweep_finished.clear()
22        self.send_command("SWEEP")
23        self.sweep_finished.wait(timeout=self.timeout.value / 1000)
24        freqs = np.array(self._sweep_freqs)
25        S11 = np.array(self._sweep_re) + 1j * np.array(self._sweep_im)
26        return freqs, S11

```

7.4 SOL Calibration at Scan Start

`scan_begin()` prompts the operator to connect each calibration standard in sequence and sweeps each one. The same closed-form ideal-standards solve from `calibration_final.py` is used inside the `_SOLTCal` class, with an option for the actual-standards mode when the standard reflection arrays are available:

Listing 8: Interactive SOL calibration in `scan_begin()` (`vna_plugin.py`)

```

1    def _run_sol_calibration(self):
2        results = {}
3        for name in ("SHORT", "OPEN", "LOAD"):
4            input(f" Attach {name} standard and press Enter...")
5            f, S11 = self._do_sweep()
6            results[name] = S11
7
8        self._freqs = f
9        self._cal = _SOLTCal(

```

```

10         freqs      = f,
11         S11m_short  = results["SHORT"],
12         S11m_open   = results["OPEN"],
13         S11m_load    = results["LOAD"],
14         delay_m      = self.delay_mm_set.value / 1000.0,
15         mode         = "ideal",
16     )

```

7.5 Per-Point Measurement: Correction, De-embedding, and NRW

At each scan position, `scan_trigger_and_wait()` fires the sweep and `scan_read_measurement()` applies the correction pipeline and writes to HDF5:

Listing 9: Correction, de-embedding, and NRW per scan point (`vna_plugin.py`)

```

1  def scan_read_measurement(self, i, l):
2      S11_raw = self._last_S11_raw
3
4      # SOLT correction + remove coax delay
5      S11_cal = self._cal.correct_and_deembed(S11_raw)
6
7      # NRW: approximate S21 from S11 (replace with measured S21
8      # if a two-port fixture is available)
9      S21 = np.sqrt(np.maximum(1.0 - np.abs(S11_cal)**2, 0.0)).astype(
10         complex)
11     L = self.sample_mm_set.value / 1000.0
12     eps_r, mu_r = _nrw(S11_cal, S21, self._freqs, L)
13
14     # Write to pre-allocated HDF5 datasets
15     ix = self._point_idx // self._scan_ny
16     iy = self._point_idx % self._scan_ny
17     self._ds_raw[ix, iy, :] = S11_raw
18     self._ds_cal[ix, iy, :] = S11_cal
19     self._ds_eps[ix, iy, :] = eps_r
20     self._ds_mu [ix, iy, :] = mu_r
21     self._h5.flush()
22
23     self._point_idx += 1

```

7.6 HDF5 Output

All data is written to the group `/scan/vna/` inside the scan HDF5 file, matching the existing scan pipeline schema:

Table 2: HDF5 datasets written by `vna_plugin.py`

Dataset	Shape	Description
<code>freqs</code>	$(N_f,)$	Frequency axis (Hz)
<code>S11_raw</code>	(N_x, N_y, N_f)	Raw measured S_{11}
<code>S11_cal</code>	(N_x, N_y, N_f)	SOLT-corrected, de-embedded S_{11}
<code>eps_r</code>	(N_x, N_y, N_f)	NRW complex permittivity
<code>mu_r</code>	(N_x, N_y, N_f)	NRW complex permeability
<code>cal/e00</code>	$(N_f,)$	SOLT error term e_{00}
<code>cal/e11</code>	$(N_f,)$	SOLT error term e_{11}
<code>cal/Delta</code>	$(N_f,)$	SOLT error term Δ_e

8 Conclusions and Future Work

8.1 Summary of Accomplishments

- Designed and simulated a Modified Colpitts Common-Base VCO covering 2.3–2.8 GHz with SMV 1247 varactor tuning
- Designed edge-coupled microstrip, rat-race, and modified rat-race directional couplers; characterized the directivity limitation arising from even/odd mode phase velocity disparity in microstrip
- Implemented SOLT one-port calibration in Python (`calibration_final.py`), supporting both ideal and actual-standards modes, with de-embedding of a coaxial delay line
- Implemented NRW material characterization in Python (`material_char.py`) using `scikit-rf` for S-parameter file handling
- Integrated both into a single `vna_plugin.py` for `Scan_Controller_Probe_Main`, following the same socket and threading architecture as `cyBot_Plugin.py`, with output written directly to the existing HDF5 scan pipeline

8.2 Future Work

- Fabricate and test VCO and coupler PCBs

- Investigate and fabricate Lange coupler and capacitive compensation to improve directivity
- Add a real S21 measurement path to the fixture so NRW does not rely on the $|S_{21}| \approx \sqrt{1 - |S_{11}|^2}$ approximation
- Integrate the full VNA signal chain (VCO $\rightarrow$ coupler $\rightarrow$ DUT $\rightarrow$ mixer $\rightarrow$ ADC $\rightarrow$ STM32H7 $\rightarrow$ PC) for end-to-end validation
- Migrate the digital processing backend from the STM32H7 to a Xilinx FPGA platform to support high-speed parallelization and expansion to N -port measurement capabilities
- Integrate ADAR2001 and ADAR2004 IC frequency multipliers and beamformers to extend the operational range into the Ka-band for high-frequency operation.